

Algorithmique 1

3. Récursivité – suite

Johan Thapper
2023–2024



Complexité

- ▶ Comment calculer la complexité d'une fonction récursive ?
- ▶ $A(n) = O(f(n))$: le nombre d'appels total à la fonction
- ▶ $B(n) = O(g(n))$: le nombre d'opérations fondamentales d'un appel
- ▶ Alors, $T(n) = O(A(n) \times B(n)) = O(f(n) \times g(n))$
- ▶ **Example** (recherche linéaire, version récursive)
 - ❖ $A(n) = O(n), B(n) = O(1) \Rightarrow T(n) = O(A(n) \times B(n)) = O(n)$
- ▶ **Example** (duplicates, version récursive)
 - ❖ $A(n) = O(n), B(n) = O(n) \Rightarrow T(n) = O(A(n) \times B(n)) = O(n \times n) = O(n^2)$

```
int find(int t[], int lo, int hi, int elt) {  
    if (lo > hi)  
        return 0;  
    else if (t[lo] == elt)  
        return 1;  
    else  
        return find(t, lo+1, hi, elt);  
}
```

```
int duplicates(int t[], int lo, int hi) {  
    if (lo > hi)  
        return 0;  
    else if (find(t, lo+1, hi, t[lo]))  
        return 1;  
    else  
        return duplicates(t, lo+1, hi);  
}
```

La pile d'exécution

- ▶ Les appels récursifs augmente la pile d'exécution
- ▶ Exemple

```
void reverse(int t[], int lo, int hi) {  
    if (lo > hi)  
        return;  
    else {  
        swap(&t[lo], &t[hi]);  
        reverse(t, lo+1, hi-1);  
    }  
}
```

reverse(t, 3, 2)
reverse(t, 2, 3)
reverse(t, 1, 4)
reverse(t, 0, 5)

- ▶ Au total, on ajoute $1+(hi-lo)/2$ appels sur la pile (division entière)
 - ❖ en plus du premier appel
- ▶ Si $hi-lo$ est grand, alors la pile devient pleine et le programme termine avec une erreur de segmentation

Récurtivité terminale

- Certaines fonctions récursives qu'on a vues retournent immédiatement après leur appel récursif

```
void reverse(int t[], int lo, int hi) {  
    if (lo > hi)  
        return;  
    else {  
        swap(&t[lo], &t[hi]);  
        reverse(t, lo+1, hi-1);  
    }  
}
```

```
int find(int t[], int lo, int hi, int elt) {  
    if (lo > hi)  
        return 0;  
    else if (t[lo] == elt)  
        return 1;  
    else  
        return find(t, lo+1, hi, elt);  
}
```

- Cela s'appelle la **récurtivité terminale**.

Élimination de l'appel terminal

- ▶ Une fonction qui utilise la **récurtivité terminale**
- ▶ Conversion à une fonction itérative :
 - au lieu de l'appel récursif,
 - ❖ mettre à jour les arguments
 - ❖ faire un saut (**goto**) au début de la fonction
- ▶ L'instruction **goto** peut être **éliminée**

```
void reverse_rec(int t[], int lo, int hi) {  
    if (lo > hi)  
        return;  
    else {  
        swap(&t[lo], &t[hi]);  
        reverse_rec(t, lo+1, hi-1);  
    }  
}
```

```
void reverse_goto(int t[], int lo, int hi) {  
start:  
    if (lo > hi)  
        return;  
    else {  
        swap(&t[lo], &t[hi]);  
        lo = lo + 1;  
        hi = hi - 1;  
        goto start;  
    }  
}
```

```
void reverse_iter(int t[], int lo, int hi) {  
    for (; lo <= hi; lo += 1, hi -= 1) {  
        swap(&t[lo], &t[hi]);  
    }  
}
```

Élimination de l'appel terminal

- ▶ Certains compilateurs peuvent éliminer l'appel terminal
- ▶ Exemple (clang sur Mac OS X, gcc sur Linux)
 - ❖ 500000 appels
 - ❖ pile d'exécution de taille 8 Mo

```
$ gcc reverse-test.c  
$ ./a.out  
first element is 0  
calling reverse_rec  
Segmentation fault: 11
```

```
$ gcc -O1 reverse-test.c  
$ ./a.out  
first element is 0  
calling reverse_rec  
first element is 999999
```

La suite de Fibonacci

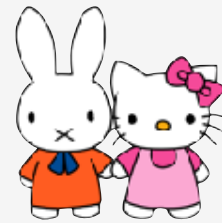
$F(n)$: le nombre de couples au mois n

$$F(0) = 0$$

$$F(1) = 1$$

$$F(n) = F(n-1) + F(n-2)$$

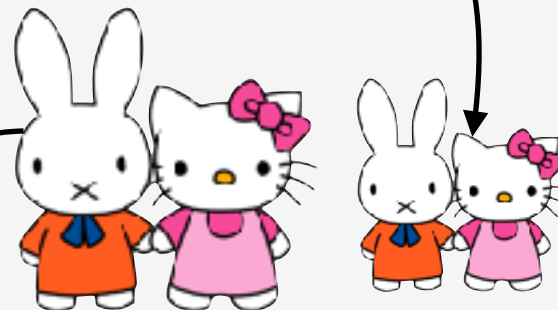
Mois 1



Mois 2



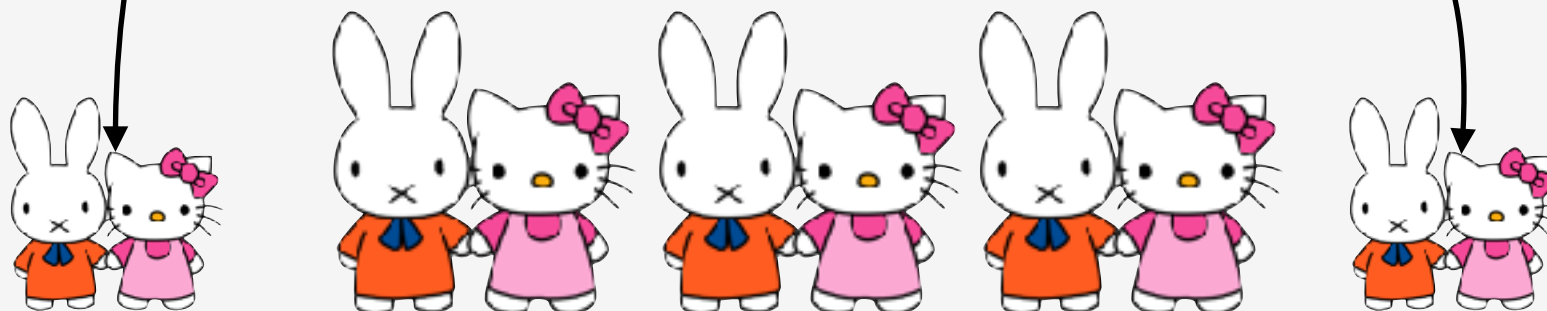
Mois 3



Mois 4



Mois 5



Leonardo Bonacci
filius Bonacci—"Fibonacci"
(1175–1250)

Fibonacci récursive

Définition par récurrence :

$$F(0) = 0$$

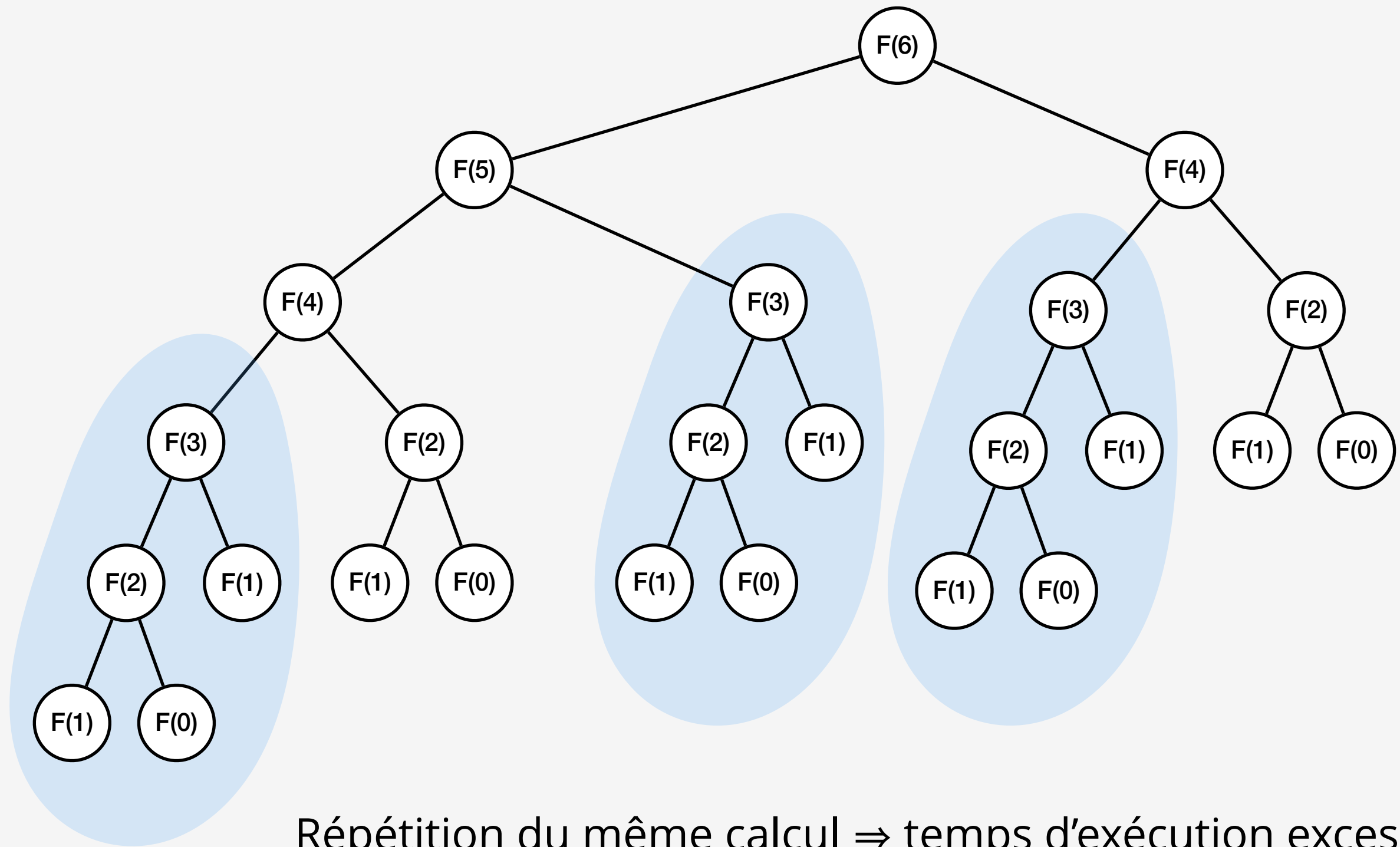
$$F(1) = 1$$

$$F(n) = F(n-1) + F(n-2) \text{ si } n > 1$$

Traduction directe en C de la définition :

```
int fibo_rec(int n) {  
    if (n == 0)  
        return 0;  
    else if (n == 1)  
        return 1;  
    else  
        return fibo_rec(n-1) + fibo_rec(n-2);  
}
```

Arbre d'appels



Temps exponentiel

- ▶ Le calcul de $F(n)$ nécessite $2F(n+1)-1$ appels à `fibonacci_rec`
 - ❖ raisonnement par récurrence
- ▶ Pour $F(6)$: 25 appels
- ▶ Pour $F(50)$: 40730022147 $\approx$ 40 milliards appels
- ▶ Pour $F(1000)$: 140660735422845631643670509754367099540362 appels
539672717465485209810174309074236393867159
484498989125223466975500898483531982176372
726530900447294212024106748242547734678222
396278746251197535380183804490490646807001
- ▶ $F(n) \approx (1.618^n)/\sqrt{5}$ donc $F(n)$ nécessite plus de 1.618^n appels
- ▶ La fonction `fibonacci_rec` est donc exponentielle et **très inefficace** !

Fibonacci it rative

- La r cursivit  est une approche “top-down” (de haut en bas) :
 - ❖ Chercher d’abord la valeur de $F(n)$
 - ❖ Pour la trouver, il faut chercher les valeurs pour des plus petites n
- Approche “bottom-up” (de bas en haut) :
 - ❖ Initialiser les valeurs les plus petites $F(0) = 0$ et $F(1) = 1$
 - ❖ Calculer ensuite les valeurs de $F(2)$, $F(3)$, ..., $F(n)$
 - ❖ Complexit  : $O(n)$ au lieu de $O(1.618^n)$...

```
int fibo_iter(int n) {  
  
    int f[MAX_SIZE];  
  
    f[0] = 0;  
    f[1] = 1;  
    int i;  
    for (i = 2; i <= n; i++)  
        f[i] = f[i-1] + f[i-2];  
    return f[n];  
}
```

i							
	0	1	2	3	4	5	6
f	0	1	1	2	3	5	

Simplification

```
int fibo_rec(int n) {  
    if (n == 0)  
        return 0;  
    else if (n == 1)  
        return 1;  
    else  
        return fibo_rec(n-1)+fibo_rec(n-2);  
}
```

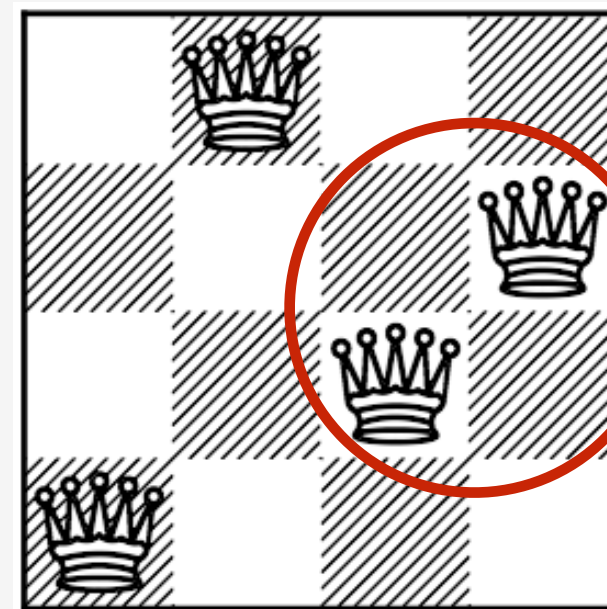
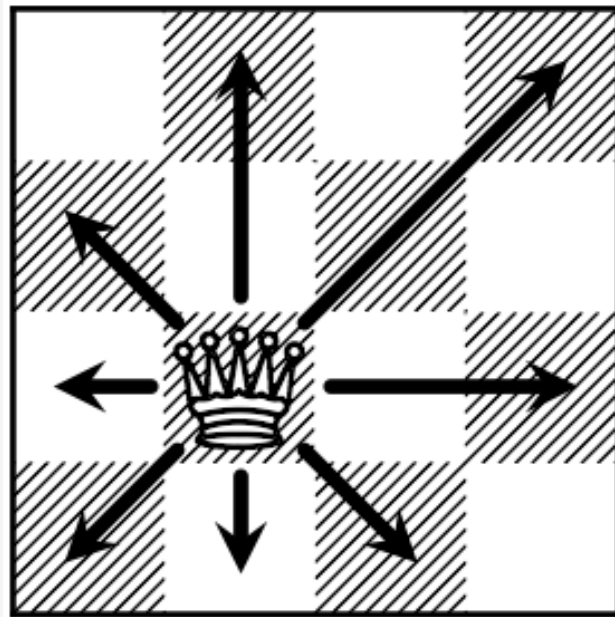
```
int fibo_iter(int n) {  
  
    int f[MAX_SIZE];  
  
    f[0] = 0;  
    f[1] = 1;  
    int i;  
    for (i = 2; i <= n; i++)  
        f[i] = f[i-1]+f[i-2];  
    return f[n];  
}
```

```
int fibo_iter_simple(int n) {  
    if (n <= 1)  
        return n;  
    int i, a, b, c;  
    a = 0;  
    b = 1;  
    for (i = 2; i <= n; i++) {  
        c = a+b;  
        a = b;  
        b = c;  
    }  
    return c;  
}
```

le tableau int f[MAX_SIZE]
n'est pas nécessaire



Exemple : 4 dames

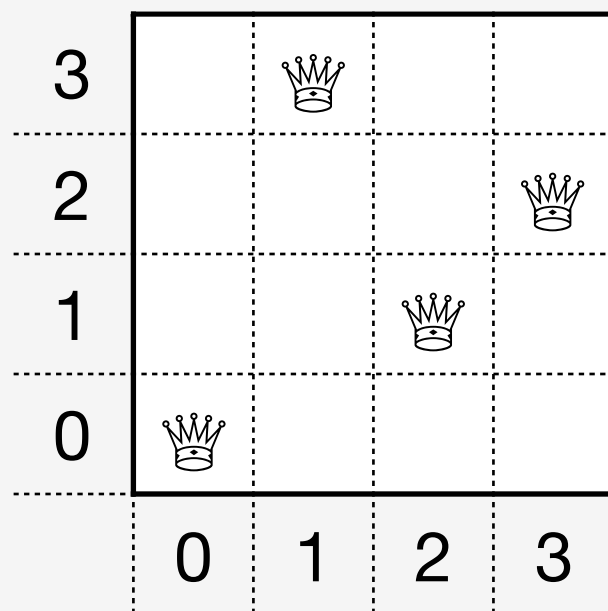


conflit

De combien de manières peut-on placer 4 dames sur un échiquier de façon à ce qu'aucune d'entre elles soit en prise ?

Représentation

- ▶ Il y a exactement une dame dans chaque colonne et rang
- ▶ Représenter l'échiquier par un tableau `int board[4]`
 - ❖ une solution à 4 dames est une permutation de $\{0,1,2,3\}$
 - ❖ mais toute permutation n'est pas une solution (voir l'exemple)



board

0	1	2	3
0	3	1	2

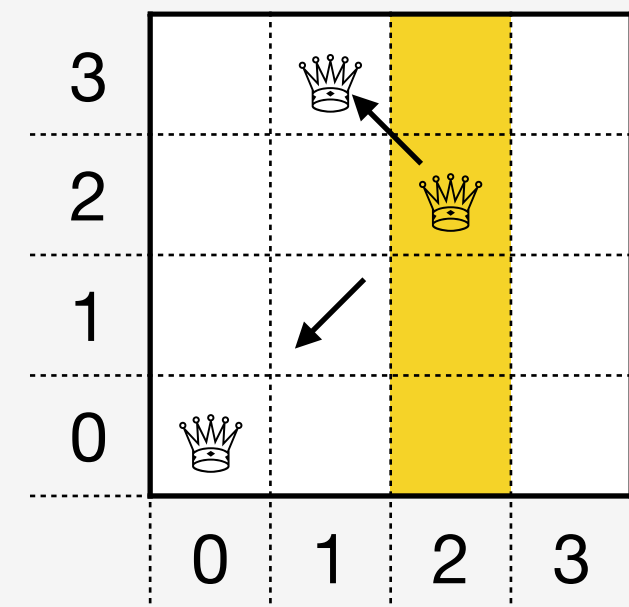
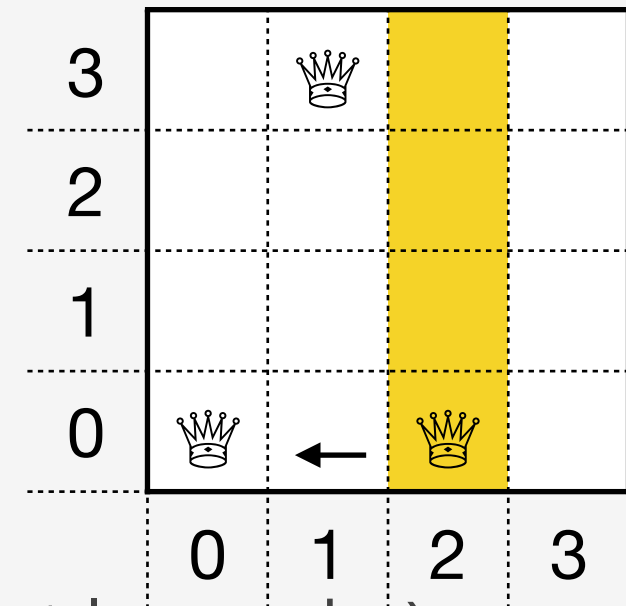
`board[1] == 3` si la dame dans **colonne 1** est placée dans le **rang 3**

Détection de conflit

- Supposer que les colonnes 0 et 1 sont remplies correctement

	0	1	2	3
board	0	3		

- Détecter une attaque horizontale
 - ❖ $\text{board}[2] == \text{board}[i]$ pour $i = 0$ ou 1
- Détecter une attaque diagonale
 - ❖ $2-i == \text{board}[2] - \text{board}[i]$ (vers le bas et la gauche)
 - ❖ $2-i == \text{board}[i] - \text{board}[2]$ (vers le haut et la gauche)



```
int conflict(int board[], int size, int pos) {  
    int i;  
    for (i = 0; i < pos; i++) {  
        /***  
        * If the queen in column i is in conflict  
        * with the queen in column pos, then  
        * return 1  
        */  
    }  
    return 0;  
}
```

Solution naïve

- ▶ 4 dames = 4 boucles imbriquées
- ▶ 8 dames = 8 boucles ...
- ▶ N dames = N boucles ?
- ▶ Solution : récursivité
 - ❖ “répétition généralisée”

```
int count = 0;
int i, i0, i1, i2, i3;
for (i0 = 0; i0 < 4; i0++)
for (i1 = 0; i1 < 4; i1++)
for (i2 = 0; i2 < 4; i2++)
for (i3 = 0; i3 < 4; i3++) {
    board[0] = i0;
    board[1] = i1;
    board[2] = i2;
    board[3] = i3;
    int board_ok = 1;
    for (i = 1; i < 4; i++)
        if (conflict(board, size, i))
            board_ok = 0;
    if (board_ok)
        count++;
}
return count;
```

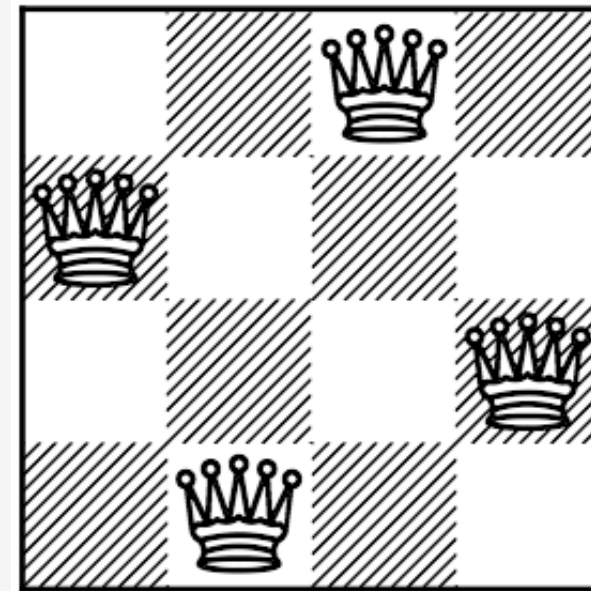
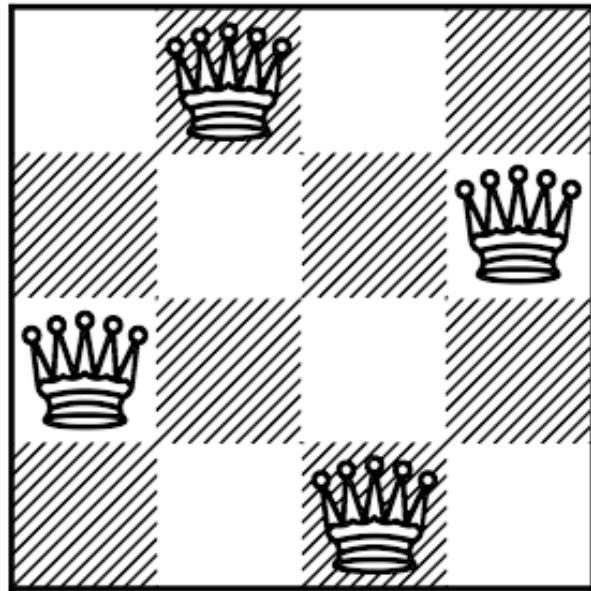
- ▶ Les tests de conflit sont à la fin, donc on effectue 4^4 tests
 - ❖ 8 dames : $8^8 = 2^{24} \approx 16,7$ millions tests
 - ❖ 10 dames : $10^{10} = 10$ milliards tests
- ▶ Amélioration : effectuer les tests dès que possible

Solution récursive

- ▶ Fonctionne pour un N (size) quelconque sans modification
- ▶ On passe comme argument la colonne de la dame à placer (pos)
- ▶ Si on a placé toutes les dames (pos == size), c'est fini, retourner 1
- ▶ Sinon, pour tout choix i de ligne
 - ❖ placer la dame à la position (pos, i)
 - ❖ s'il n'y a pas de conflit, on place la dame suivante

```
int queens(int board[], int size, int pos) {  
    if (pos == size) {  
        return 1;  
    }  
    else {  
        int i, count = 0;  
        for (i = 0; i < size; i++) {  
            board[pos] = i;  
            if (!conflict(board, size, pos))  
                count = count + queens(board, size, pos+1);  
        }  
        return count;  
    }  
}
```

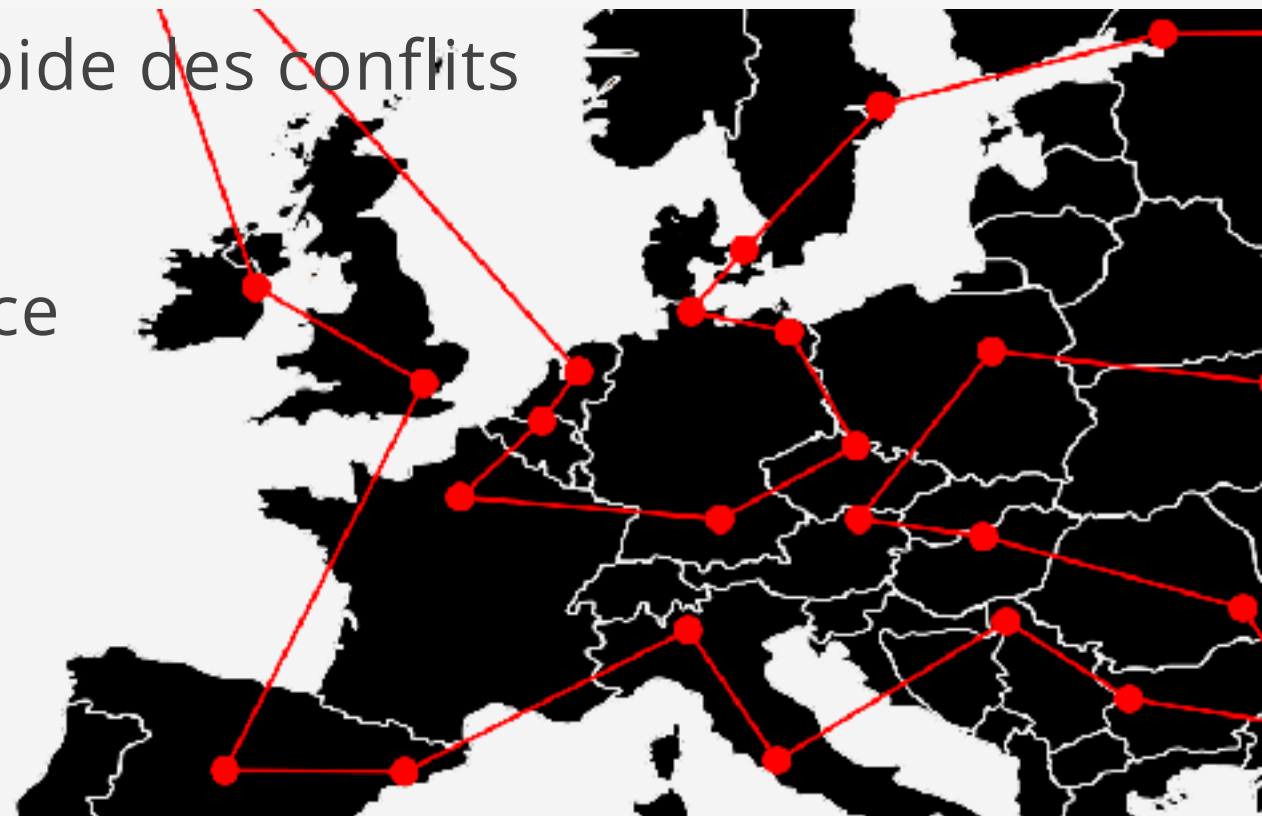
4 dames, 8 dames, ...



- ▶ Il y a 2 solutions possibles pour 4 dames
- ▶ Il y a 92 solutions pour 8 dames
- ▶ La fonction récursive trouve 14 772 512, le nombre de solutions pour 16 dames, en ≈ 3 minutes
- ▶ Il y a 234 907 967 154 122 528 solutions pour 27 dames (sept 2016)
<https://github.com/preusser/q27>
<https://oeis.org/A000170>
- ▶ Inconnu pour 28 dames (exercice bonus)

Backtracking

- ▶ La solution récursive à N dames se base sur un principe algorithmique général qui s'appelle **backtracking**, un type de recherche exhaustive
 - ❖ découper le problème en petits morceaux (par exemple, board[i])
 - ❖ pour toute possibilité d'affectation au **morceau i**
 - ❖ s'il n'y a pas de conflits, essayer d'affecter le **morceau i+1**
 - ❖ quand toutes les possibilités pour **i** sont testées, on remonte à **i-1**
- ▶ N morceaux différents et M possibilités pour chaque morceau
⇒ M^N possibilités au pire des cas, c'est donc **exponentiel** !
- ▶ L'efficacité dépend de la détection rapide des conflits
- ▶ Autres problèmes :
 - ❖ Problème du voyageur de commerce
 - ❖ Problème du sac à dos
 - ❖ Sudoku
 - ❖ Labyrinthe



Résumé

- ▶ Fonction récursive
 - ❖ la plus naturelle pour concevoir une fonction à partir d'une définition par récurrence
 - ❖ coût auxiliaire : gestion de la pile d'exécution
 - ❖ à éviter si la fonction est linéaire et/ou facile à écrire en itérative
- ▶ Récursivité terminale
 - ❖ forme particulière de la récursivité qui peut être éliminée (automatiquement)
- ▶ Complexité
 - ❖ estimation du nombre d'appels et leur complexité
- ▶ Très inefficace s'il y a des chevauchements entre les calculs partiels
 - ❖ une fonction itérative (bottom-up) est plus efficace (mais cache la structure de la définition par récurrence)

Résumé

- ▶ Backtracking
 - ❖ approche générale à un grand nombre de problèmes
 - ❖ peut être implanté par la récursivité
 - ❖ peut être exponentiel dans le pire des cas